

README

Project Title

Arduino Data Logger: Sensor to SD Card

Project Overview

This project demonstrates a sensor data logging system built in the Wokwi Arduino simulation environment. An ESP32 microcontroller reads temperature and humidity from a DHT22 sensor and ambient light level from an LDR, timestamps each reading using a DS1307 real-time clock module, and stores the readings as CSV entries on an SD card via the SPI interface.

Objective

Collect sensor data (temperature and light, plus humidity) at a fixed sampling interval and save timestamped entries to an SD card in CSV format, producing a log of at least 100 samples.

Software & Libraries

- Arduino IDE / Wokwi Simulator
- DHT.h (temperature & humidity sensor)
- Wire.h (I2C communication)
- RTCLib.h (DS1307 real-time clock)
- SPI.h (SPI communication)
- SD.h (SD card read/write)

Arduino Sketch

```
#include <Arduino.h>
#include <DHT.h>
#include <Wire.h>
#include <RTCLib.h>
#include <SPI.h>
#include <SD.h>

// -----
// Pin definitions
// -----
#define DHT_PIN 4
#define DHT_TYPE DHT22

#define LDR_PIN 34
#define SD_CS 5

// -----
// Objects
// -----
DHT dht(DHT_PIN, DHT_TYPE);
RTC_DS1307 rtc;
File dataFile;
```

```

const unsigned long SAMPLE_INTERVAL = 1000;    // 1 sample per second
unsigned long lastSampleTime = 0;
unsigned long sampleCount = 0;

// -----
// Setup
// -----
void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println();
  Serial.println("=====");
  Serial.println("  SENSOR DATA  LOGGER");
  Serial.println("=====");

  // Start DHT sensor
  dht.begin();

  // Start RTC
  if (!rtc.begin()) {
    Serial.println("Couldn't find RTC");
  }
  if (!rtc.isrunning()) {
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
  }

  // Start SD card
  if (!SD.begin(SD_CS)) {
    Serial.println("SD card initialization failed!");
    return;
  }
  Serial.println("SD card initialized.");

  // Create CSV file and write header if it does not exist
  if (!SD.exists("/log.csv")) {
    dataFile = SD.open("/log.csv", FILE_WRITE);
    if (dataFile) {
      dataFile.println("Date,Time,Temperature_C,Humidity_%,Light");
      dataFile.close();
    }
  }
}

// -----
// Loop
// -----
void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - lastSampleTime >= SAMPLE_INTERVAL) {
    lastSampleTime = currentMillis;

    DateTime now = rtc.now();
    float temperature = dht.readTemperature();
    float humidity    = dht.readHumidity();
    int   lightLevel = analogRead(LDR_PIN);

    if (isnan(temperature) || isnan(humidity)) {
      Serial.println("Failed to read from DHT sensor!");
      return;
    }

    char dateStr[11];

```

```

sprintf(dateStr, "%04d-%02d-%02d", now.year(), now.month(), now.day());
char timeStr[9];
sprintf(timeStr, "%02d:%02d:%02d", now.hour(), now.minute(), now.second());

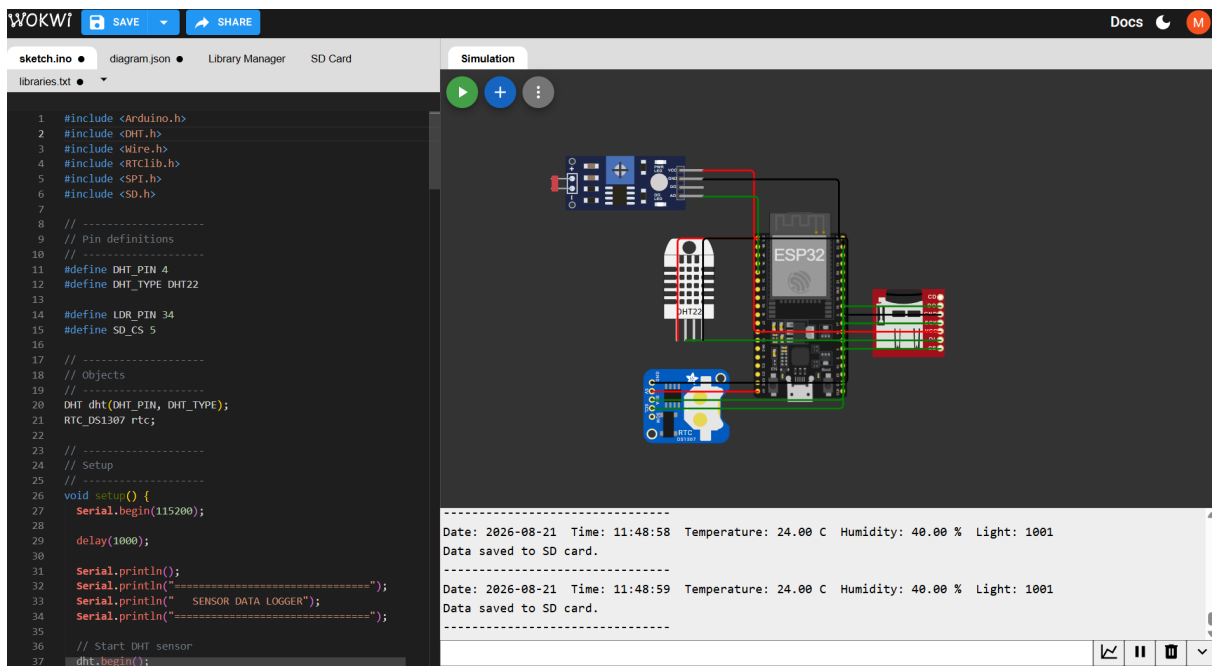
Serial.println("-----");
Serial.print("Date: ");      Serial.print(dateStr);
Serial.print(" Time: ");     Serial.print(timeStr);
Serial.print(" Temperature: "); Serial.print(temperature);
Serial.print(" C Humidity: "); Serial.print(humidity);
Serial.print(" % Light: ");   Serial.println(lightLevel);

dataFile = SD.open("/log.csv", FILE_WRITE);
if (dataFile) {
  dataFile.print(dateStr); dataFile.print(",");
  dataFile.print(timeStr); dataFile.print(",");
  dataFile.print(temperature); dataFile.print(",");
  dataFile.print(humidity); dataFile.print(",");
  dataFile.println(lightLevel);
  dataFile.close();
  sampleCount++;
  Serial.println("Data saved to SD card.");
} else {
  Serial.println("Error opening log.csv");
}
}
}

```

Simulation Circuit

The circuit was built and tested in Wokwi. The ESP32 is wired to a DHT22 temperature/humidity sensor (data pin 4), an LDR light sensor on ADC pin 34, a DS1307 RTC module over I2C, and a micro SD card module over SPI (CS on pin 5).



Working Principle

On startup the program initializes the DHT22 sensor, the DS1307 RTC, and the SD card in SPI mode, creating log.csv with a header row if it does not already exist. In the main loop, every 1000 ms the program reads the current temperature and humidity from the DHT22, reads the analog light level from the LDR, and fetches the current date and time from the RTC. Each reading is printed to the Serial Monitor and appended as a new timestamped row to log.csv on the SD card, then the file is closed to ensure the data is flushed to storage.

Sampling Rate & Power Considerations

- **Sampling rate:** One sample per second (SAMPLE_INTERVAL = 1000 ms), giving 100 logged samples in roughly 100 seconds of runtime. The interval is easily adjustable depending on how fast the monitored conditions change.
- **SD write overhead:** Opening and closing the file on every write costs extra time and a small current spike compared to keeping the file open, but it guarantees each sample is safely flushed to the card even if power is lost between writes.
- **Power saving options:** For battery-powered deployments, the sampling interval can be lengthened, sensor reads/SD writes can be batched, and the ESP32 can be put into deep sleep between samples, waking on a timer to take the next reading and drastically reducing average current draw.
- **Sensor/RTC power:** The DHT22 has a minimum ~2 second interval between reliable reads; the DS1307 draws negligible current and keeps time via its backup battery even when the main board is powered down.

Output

Serial Monitor (per sample): Date: 2026-08-21 Time: 11:48:58 Temperature: 24.00 C Humidity: 40.00 % Light: 1001, followed by "Data saved to SD card."

SD card (log.csv): Date,Time,Temperature_C,Humidity_%,Light — followed by one row per sample, e.g. 2026-08-21,11:48:58,24.00,40.00,1001

Deliverables

- 1 CSV file (log.csv) from the SD card showing at least 100 samples.
- 2 Arduino Sketch (.ino)
- 3 Code and explanation of sampling rate and power considerations (above).
- 4 Simulation Screenshot
- 5 README PDF

Conclusion

The project successfully demonstrates a working sensor data logger that reads temperature, humidity, and light data on a fixed schedule, timestamps each reading using an RTC, and persists the results as a CSV log on an SD card — suitable as a foundation for longer-term environmental monitoring.